



United International University

Department of Computer Science and Engineering

CSI 309/CSE 4509: Operating System Concepts/Operating Systems

Final Examination : Spring 2026

Total Marks: 40 Time: 2 Hours

Answer all the 4 questions. Numbers to the right of the questions denote their marks.

Any examinee found adopting unfair means will be expelled from the trimester / program as per UIU disciplinary rules.

1. Answer the following questions:

- (a) How can imposing a certain ordering on resources prevent circular wait?? Illustrate your answer with a simple example involving two mutex locks. [2]
- (b) Draw a resource allocation graph for the following C code and comment if a deadlock can occur or not. [4]

```
#define NUM_MUTEXES 5
#define NUM_THREADS 4
pthread_mutex_t mutexes[NUM_MUTEXES];

void* worker(void* arg) {
    int id = *((int*)arg);

    int first = id % NUM_MUTEXES;
    int second = (id + 2) % NUM_MUTEXES;
    pthread_mutex_lock(&mutexes[first]);
    sleep(2);
    pthread_mutex_lock(&mutexes[second]);

    pthread_mutex_unlock(&mutexes[second]);
    pthread_mutex_unlock(&mutexes[first]);
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS] = {1, 2, 3, 4};

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, worker, &thread_ids[i]);
    }
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    return 0;
}
```

- (c) Consider the following system snapshot with five threads (T_0 to T_4) and four resource types (A, B, C, D).

Thread	Allocation				Max			
	A	B	C	D	A	B	C	D
T0	0	1	0	1	0	3	1	2
T1	2	1	0	0	4	2	4	1
T2	1	1	1	0	3	2	3	1
T3	0	2	0	1	0	3	1	2
T4	1	0	2	0	3	1	3	2

Table 1: Allocation and Max Matrices

Determine whether the system is in a safe state or not if the available resources are $\{A, B, C, D\} = \{0, 1, 1, 1\}$. If the state is safe, illustrate a safe sequence in which the threads may complete. Otherwise, calculate the minimum number of additional available resources required to ensure the execution of all threads. [4]

2. An operating system is designed for a machine with **512 MB of RAM** and a **32-bit virtual address space**. The OS uses **4 KB pages** and each **Page Table Entry (PTE)** is **4 bytes**.
 - (a) What is the size of the page offset in bits? [1]
 - (b) How many bytes are there in a page? [1]
 - (c) What is the size of the physical address in bits? [2]
 - (d) How many virtual pages are there in the virtual memory? [1]
 - (e) How much memory does a single-level page table consume per process? [2]
 - (f) Design a multi-level page table for this system. Clearly state how the virtual address bits are divided among the levels and offset. [3]

3. Sam uses a mini computer with only **16 KB of physical RAM**. The operating system supports an **18-bit virtual address space** and uses a **page size of 4 KB**. Sam is running a program called *PageRunner*, which generates a sequence of virtual page references during its execution.
 - (a) Can Sam's mini computer store the entire virtual address space in its physical RAM at the same time? Show all necessary calculations to justify your answer. [2]
 - (b) While executing *PageRunner*, the following sequence of virtual page references is generated:

2, 5, 1, 2, 5, 7, 10, 2, 12, 15, 6, 12, 16, 6

Assume that the page table is initially empty and **Least Recently Used (LRU)** page replacement policy is used. The number of page frames available in physical memory is determined by the given RAM size and page size.

Simulate the page-reference sequence and determine the contents of physical memory after each page reference. Then calculate the total number of page hits and page misses. [3+1]
 - (c) Using your results from **part (b)**, calculate the **total time required** to fetch all memory references generated by *PageRunner*.

Assume that in case of **page hit**, the page is accessed from RAM and it takes **10 ns**, whereas a **page miss/fault** requires fetching the page from disk and takes **2 μs**. Ignore all other overheads. [4]

4. While Gyomei and Muichiro were running through the infinity castle purposelessly, they started discussing disk allocation. Gyomei proposes that all the files of their 2TB SSD should be allocated using **FAT (Linked list allocation with a table in memory)** method. But Muichiro said, this approach might create some problems. Instead, he suggests that only the files that need random access (approximately 2GB of data) should be allocated using **FAT**, other files should be allocated using **Regular Linked List allocation**. A random demon in the background shouted, "Why not use **contiguous allocation** for all files?"
 - (a) What is the purpose of the boot block and super block? [2]
 - (b) What problem can occur in Gyomei's approach? [2]
 - (c) Between the proposed approaches of Gyomei and Muichiro, which one is overall better? Justify your answer under the following criteria: File access time, growth flexibility and RAM utilization. [3]
 - (d) Do you think the demon's suggestion should be taken? Justify your answer. [2]
 - (e) State under which circumstances, the demon's suggested method can be used. [1]